

Instituto Tecnológico de Costa Rica
Escuela de Computadores
CE 1106: Paradigmas de Programación

Tarea de Programación
Paradigma Funcional, Recursividad y Estructura de Datos
BusCEminas

Profesor:
Marco Rivera Meneses

Integrantes:
Gabriel Soto López – 2024178797
Luis Castro Montenegro – 2013083843
Saddy Guzman Rojas – 2023088184

Septiembre, 2025

Índice

Introducción.....	3
Descripción de las funciones implementadas	4
Ejemplificación de las estructuras de datos desarrolladas.....	6
Descripción detallada de los algoritmos desarrollados	7
Problemas sin solución.....	11
Plan de actividades realizadas	11
Problemas encontrados.....	12
Conclusiones	13
Recomendaciones	13
Bibliografía.....	14
Bitácora.....	15

Introducción

El presente documento trata el desarrollo de **BusCEMinas**, una implementación del juego clásico de lógica buscaminas utilizando el lenguaje de programación Racket bajo el paradigma funcional.

El Buscaminas es un juego donde el jugador debe descubrir todas las celdas de un tablero que no contienen minas, basándose en pistas numéricas que indican la cantidad de minas adyacentes. La implementación de este juego en un paradigma funcional representó un desafío interesante, ya que requiere manejar estado mutable (el progreso del juego) mientras se mantienen los principios de inmutabilidad y transparencia referencial característicos de este paradigma.

Esta documentación describe detalladamente las decisiones de diseño, algoritmos implementados, estructura de datos utilizada y los desafíos encontrados durante el desarrollo, así como las soluciones adoptadas para superarlos.

Descripción de las funciones implementadas

Módulo: busCEminas.rkt (Lógica del Juego)

1. Función: **crear_tablero_vacio**
 - a. Descripción: Crea un tablero vacío de dimensiones dadas, inicializando con ceros.
 - b. Parámetros:
 - i. filas: Número de filas del tablero.
 - ii. Columnas: Número de columnas del tablero.
 - c. Retorna: Una lista de listas representando el tablero vacío.
2. Función: **calcular_bombas**
 - a. Descripción: Calcula el número de minas según el nivel de dificultad y el tamaño del tablero.
 - b. Parámetros:
 - i. filas: Número de filas.
 - ii. columnas: Número de columnas.
 - iii. nivel: Nivel de dificultad (1: Fácil, 2: Medio, 3: Difícil).
 - c. Retorna: Número entero de minas a colocar.
3. Función: **generar_posiciones_unicas**
 - a. Descripción: Genera una lista de posiciones únicas donde se colocarán las minas.
 - b. Parámetros:
 - i. fila: Número de filas.
 - ii. columnas: Número de columnas.
 - iii. k: Número de minas a colocar.
 - c. Retorna: Lista de pares (fila columna) únicos.
4. Función: **crear_tablero_completo_desde_bombas**
 - a. Descripción: Construye el tablero completo colocando minas y calculando números de minas adyacentes.
 - b. Parámetros:
 - i. filas: Número de filas.
 - ii. columnas: Número de columnas.
 - iii. pos_bombas: Lista de posiciones de minas.
 - c. Retorna: Tablero completo con minas y conteos adyacentes.
5. Función: **expandir_desde**
 - a. Descripción: Expande recursivamente las celdas vacías (con valor 0) a partir de una celda dada.
 - b. Parámetros:
 - i. tablero: Tablero actual.
 - ii. revelado: Matriz booleana que indica celdas ya reveladas.
 - iii. f, c: Coordenadas de la celda desde donde expandir.
 - c. Retorna: Nueva matriz “revelado” actualizada.

6. Función: **es_victoria**
 - a. Descripción: Verifica si todas las celdas sin minas han sido reveladas.
 - b. Parámetros:
 - i. tablero: Tablero con minas y números.
 - ii. revelado: Matriz booleana de celdas reveladas.
 - c. Retorna: True(#t) si se gana, False(#f) en caso contrario.
7. Función: **revelar_bombas**
 - a. Descripción: Revela todas las minas en el tablero (al perder).
 - b. Parámetros:
 - i. tablero: Tablero con minas.
 - ii. revelado: Matriz de celdas reveladas.
 - c. Retorna: Nueva matriz “revelado” con todas las minas visibles.

Módulo: busCEminas_UI_rkt (Interfaz gráfica)

1. Clase: **tablero-canvas%**
 - a. Descripción: Canvas personalizado que dibuja el tablero y maneja eventos de clic.
 - b. Método: set_tablero
 - i. Descripción: Establece un nuevo tablero y reinicia el estado del juego.
 - ii. Parámetros:
 1. t: Tablero generado por la lógica.
 - c. Método: revelar_desde
 - i. Descripción: Revela una celda y expande si es cero. Verifica fin del juego.
 - ii. Parámetros:
 1. f, c: Coordenadas de la celda.
 - d. Método: toggle_bandera
 - i. Descripción: Marca/desmarca una celda con bandera.
 - ii. Parámetros:
 1. f, c: Coordenadas de la celda.
2. Función: **draw_tablero**
 - a. Descripción: Dibuja el tablero en el canvas según el estado de revelado y banderas.
 - b. Parámetros:
 - i. dc: Contexto del dibujo.
 - ii. tablero: Tablero lógico.
 - iii. revelado: Matriz de celdas reveladas.
 - iv. banderas: Matriz de celdas con bandera.

3. Función: **on-generar**

- a. Descripción: Callback del botón “Generar”. Crea un nuevo tablero según los parámetros de entrada.
- b. Parámetros:
 - i. `_btn`: Botón (no usado).
 - ii. `_evt`: Evento (no usado).

Ejemplificación de las estructuras de datos desarrolladas

Estructura del tablero lógico (con minas y números)

- **Formato:** Lista de listas de números entero.
- **Descripción:** Cada celda es un entero:
 - 0-8: Número de minas adyacentes (0 = vacío).
 - 9: Representa una mina.
- **Ejemplo** (3x3 con 1 mina en (1,1)):


```
((1 1 1)
 (1 9 1)
 (1 1 1))
```

Estructura de posiciones (coordenadas)

- **Formato:** Lista de pares (fila columna).
- **Descripción:** Usada para representar minas, vecinos, etc...
- **Ejemplo** (3 minas en un tablero 4x4):


```
((0 2) (1 1) (3 3))
```

Matriz booleana de celdas reveladas

- **Formato:** Lista de listas booleanos.
- **Descripción:** Indica si cada celda ha sido descubierta (`#t`) o no (`#f`).
- **Ejemplo** (para tablero 2x2, solo revelada (0,0)):


```
((#t #f)
 (#f #f))
```

Matriz booleana de banderas

- **Formato:** Igual que la matriz de reveladas.
- **Descripción:** Indicada si hay una bandera en la celda.
- **Ejemplo** (bandera en (0,1) y (1,0)):


```
((#f #t)
 (#t #f))
```

Estructura de conteo de minas adyacentes (durante construcción)

- **Formato:** Lista de elementos ((fila columna) conteo).
- **Descripción:** Usada internamente por “`contar_frecuencias`” para acumular conteos antes de aplicarlos al tablero.
- **Ejemplo** (2 celdas con minas adyacentes):
 $((0\ 1)\ 2)$
 $((1\ 0)\ 1)$

Lista de offsets para vecinos

- **Formato:** Lista de desplazamientos (df dc).
- **Descripción:** Usada para calcular celdas adyacentes:
- **Ejemplo** (8 vecinos):
 $((-1\ -1)\ (-1\ 0)\ (-1\ 1))$
 $(0\ -1)\ (0\ 1)$
 $(1\ -1)\ (1\ 0)\ (1\ 1)$

Descripción detallada de los algoritmos desarrollados

A continuación, se describirá, en orden lógico, los algoritmos principales del programa `busCEminas`. Mediante la interfaz el usuario puedes ingresar el número de filas, columnas y el nivel, de 1 a 3. Con esto se determina el número de bombas necesarias.

Cada nivel asigna un porcentaje máximo de bombas baso en un porcentaje por nivel, nivel 1 corresponde a un 10%, el 2 a 15% y 3 a 20%. Para esto se usa la función `calcular_bombas(filas columnas nivel)` donde cada nivel usa las funciones `num_por_nivel(nivel)` y `den_por_nivel(nivel)` para obtener el numerador y denominador que representan el porcentaje. Con estos se hace la división con la función nativa `quotient` y se multiplica por el total de celdas. En casos donde el número de bombas sea mayor al total se coloca solo una.

Posteriormente, el programa genera las posiciones de las bombas de forma aleatoria mediante la función `generar_posiciones(filas, columnas)` la cual mediante distintas funciones auxiliares genera una fila y columna aleatoria dentro del rango de la matriz y se guarda como una lista dentro de otra, se usa una función que permite validar si una de las pociones ya esta dentro de la lista principal de posiciones para volver a generar una nueva antes de reducir el contador obtenido del número de bombas.

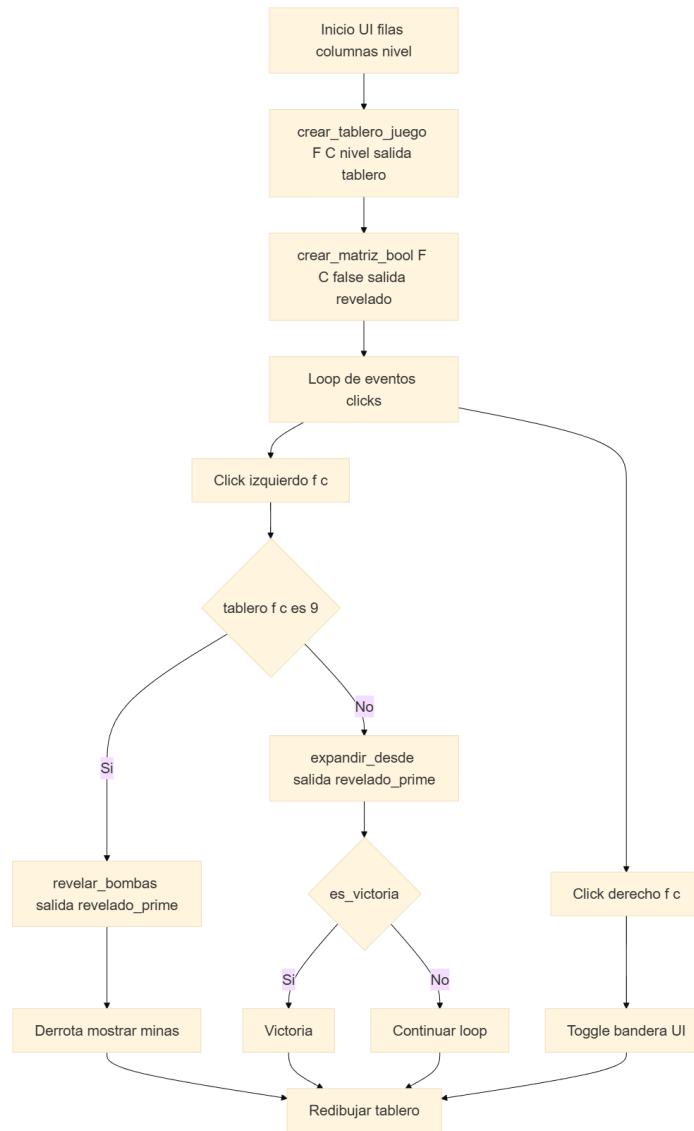
El programa se basa en dos estructuras para su funcionamiento, la matriz que representa el tablero y otra que representa que celdas del tablero han sido reveladas. El programa usa la función `crear_tablero_completo(filas, columnas, pos_bombas)`, donde este último corresponde a la lista de posiciones obtenidas en el paso anterior.

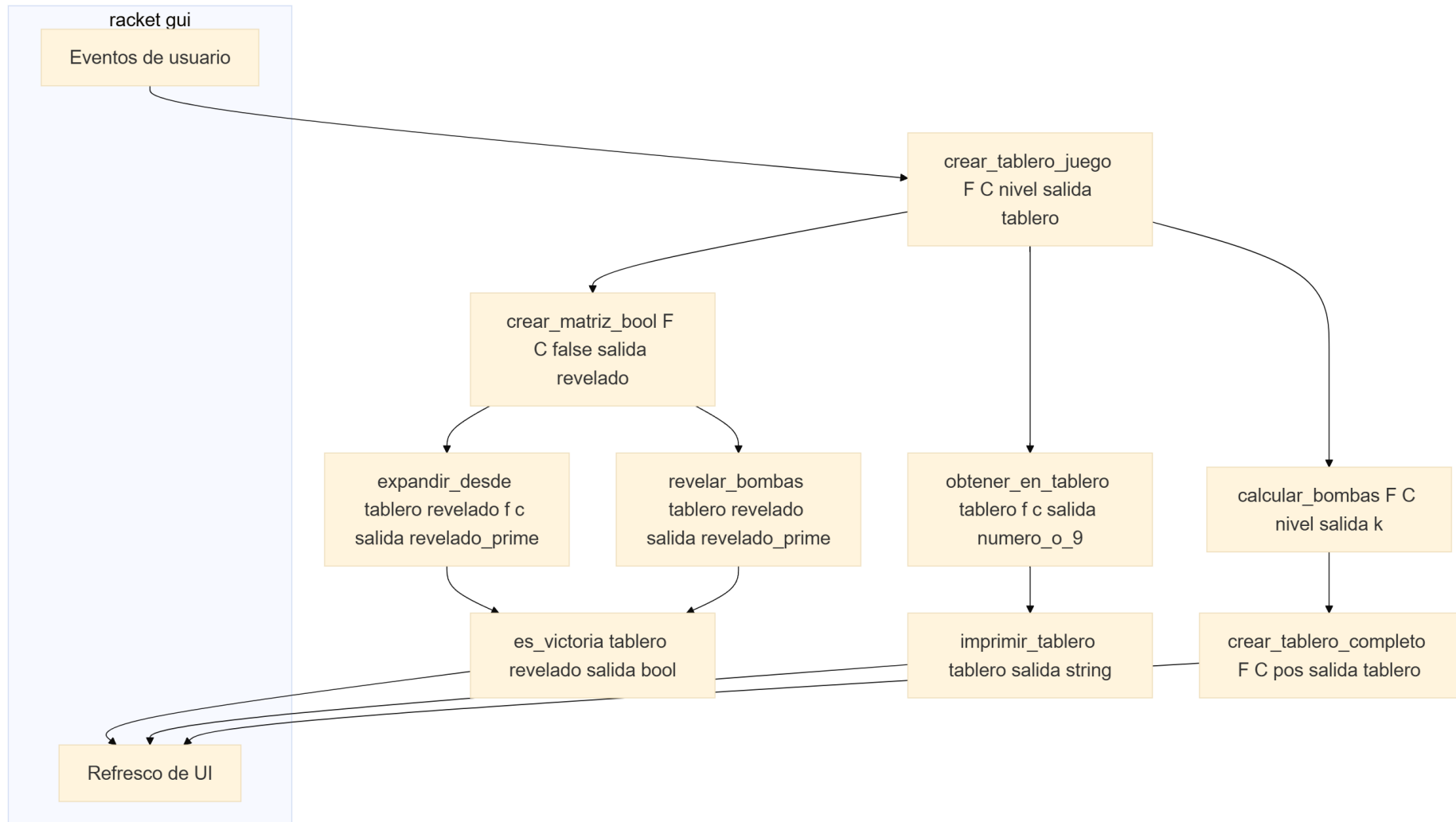
Primero se crea la matriz del tamaño seleccionado por el usuario desde la interfaz, esta matriz esta conformada solo por ceros, luego se usa una función llamada **colocar_bombas(fila, col, pos_bombas)**, donde se genera una nueva matriz que sustituye el 0 con un 9, para representar bombas, en las posiciones de la lista.

Con las bombas colocadas se usa la función **vecinos_de_bombas(fila, col, pos_bombas)** que utiliza la vecindad de Moore de 8 vecinos, mediante una lista de tuplas enteras que se usan para operar sobre una posición de **pos_bombas** y acceder a sus vecinos y agregarlos a una lista. Se usa una función llamada **filtrar_no_bombas(fila, col)** para excluir de la lista de vecinos las posiciones con bomba.

Con la lista de vecinos, posiciones, se usa **contar_frecuencias(posiciones)** para contar cuantas veces esta una posición en la lista, y con **incrementar_conteo** suma 1 a la posición en una nueva lista conformada por la posición y el contador. La función **aplicar_conteos(fila, col)** actualiza esa posición de la matriz con el contador resultante de las funciones anteriores.

Lo anterior corresponde a la lógica para la creación del tablero, a nivel del programa en general la interfaz llama a la función **crear_tablero_juego(fila, col, nivel)** la cual utiliza la lógica descrita anteriormente para generar la matriz y poder mostrarla.





Una vez que se tiene la matriz se deben ocultar las celdas para que el usuario puede jugar al dar click y revelar una. Para lograr este comportamiento se utiliza otra matriz booleana del mismo tamaño que el tablero para representar si una esta esta revelada o no, mediante la función **crear_matriz_bool(fila, col, bool)** y las funciones auxiliares **obtener_en_matriz_bool** y **reemplazar_en_matriz_bool** que se encargan de leer el estado de una celda y actualizarlo cuando se revela.

Al revelar una celda si en la matriz es un 0 se expande recursivamente revelando su frontera de vecinos, usando Moore, donde si encuentra un número distinto a 0 se detiene. Para esto se usan las funciones **expandir_desde** y **expandir_vecinos** las cuales utilizan las dos matrices para poder determinar cuando detenerse y que revelar.

Al momento de revelar una celda mediante **revelar_bombas** se determina si esa posición es un 9, donde si es la interfaz notifica al usuario que ha perdido. después de revelar una celda que no es bomba se ejecuta **es_victoria** que se encarga de verificar si todas las celdas que están reveladas corresponden a todas las posiciones que no son bombas. Para esto recorre ambas matrices simultáneamente y para cualquier valor que no sea 9 si no esta revelada retorna falso, si todas están retorna verdadero y el programa notifica al usuario.

Problemas sin solución

No hubo problemas técnicos sin solución al final del desarrollo, todas las funcionalidades principales especificadas en los requerimientos fueron implementados de forma exitosa, el juego es funciona y estable para los rangos de tamaño de malla de 8x8 hasta 15x15

Plan de actividades realizadas

Actividad	Descripción	Tiempo Estimado	Responsable	Entrega
Análisis inicial	Analizar el documento, comprender que se requiere, trazar plan.	3 horas	Todos	06/09
Diseño de estructuras de datos	Definir representación del tablero, estado de celdas, y flujo del juego	2 horas	Luis	07/09
Implementación de lógica básica	Crear tablero vacío, generar minas, calcular adyacentes	5 horas	Saddy	09/09
Algoritmo de expansión recursiva	Implementar revelado de celdas vacías (cero) y sus vecinas	4 horas	Gabriel	11/09
Interfaz gráfica básica	Desarrollar el algoritmo para calcular la posición en memoria de un estudiante basado en su índice y mostrar sus datos.	6 horas	Luis	14/09
Integración lógica-UI	Adaptar las rutinas de conversión y visualización para soportar 2 decimales en las notas	4 horas	Saddy	16/09

Actividad	Descripción	Tiempo Estimado	Responsable	Entrega
Marcado con banderas	Investigar e implementar la aritmética necesaria para aumentar la precisión a 4 y 5 decimales. Resolver problemas de precisión y representación.	3 horas	Luis	18/09
Detección de victoria/derrota	Implementar los algoritmos para calcular el promedio, nota máxima, mínima, porcentaje y aprobados/reprobados	2 horas	Gabriel	22/09
Depuración y pruebas	Implementar y depurar el algoritmo de ordenamiento (burbuja/selección) para ordenar el arreglo de estudiantes basado en su nota	6 horas	Gabriel	25/09
Documentación técnica	Integrar todos los módulos, realizar las pruebas exhaustivas de todas las funciones corregir errores encontrados	4 horas	Todos	25/09
Manual de usuario	Redactar la documentación, describir algoritmos, estructuras y decisiones de diseño.	1 h	Saddy	25/09

Problemas encontrados

- **Problema:** Lista de posiciones para las bombas al integrar con la interfaz.

Descripción: Inicialmente se desarrolló la matriz del tablero únicamente con validaciones en consola, donde la lista de posiciones no se estaba generando aleatoriamente. Se considero que la interfaz se encargara de generar las posiciones tras calcular el número de bombas y que lo pasara, pero esta lógica debía estar en el programa principal.

Solución: Se agregaron funciones para obtener las posiciones desde la lógica, donde utilizando el total de bombas como contador se van generando posiciones y se agregan a una lista. Mediante una función auxiliar se valida si la posición generada es repetida para seguir generando hasta que no lo sea.

- **Problema:** Conteo de vecinos de bomba erróneo.

Descripción: Al contar cuantas bombas vecinas tiene una posición se sobrescribían las bombas por no excluir estas. Además no todas las salidas de conteo eran correctas.

Solución: Se excluyeron las bobas y además se separaron responsabilidades para asegurar que el conteo fuera correcto y poder identificar mejor donde estaba el error de la sumatoria.

Conclusiones

El desarrollo del sistema BusCEMinas en Racket utilizando el paradigma de programación funcional representó un desafío significativo que permitió aplicar y comprender mejor los conceptos fundamentales de este paradigma, tales como la inmutabilidad de datos, la recursividad como mecanismo principal de iteración, y la composición de funciones puras. Se logró cumplir con la mayoría de los objetivos planteados, implementando un juego completamente funcional con generación aleatoria de tableros, revelado recursivo de celdas, marcado con banderas y detección automática de victoria y derrota. El uso de listas de listas para representar el tablero y matrices booleanas paralelas para el estado del juego resultó funcionar muy bien para el manejo funcional de los datos. Sin embargo, se encontraron complicaciones en la optimización del algoritmo de expansión recursiva para tableros grandes, donde el crecimiento de la pila de llamadas representó un límite.

Recomendaciones

1. Separar la lógica de la interfaz grafica: Mantener archivos distintos para el código del juego (busCEminas.rkt) y la interfaz gráfica (interfaz.rkt). La lógica debe ser independiente y poder funcionar sin la UI.
2. Probar las funciones con casos pequeños primero: Antes de integrar, verificar cada función con tableros de 2x2 o 3x3. Por ejemplo, probar que `expandir_desde` funciona correctamente en un tablero con un área vacía pequeña.
3. Documentar los tipos de datos que espera cada función: Especificar claramente qué formato deben tener los parámetros. Por ejemplo: (`crear_tablero_juego` filas columnas nivel) donde filas y columnas son números enteros positivos, nivel es 1, 2 o 3.
4. Manejar los errores de entrada del usuario: Validar que los valores ingresados en la interfaz sean números válidos antes de procesarlos. Mostrar mensajes claros cuando los datos son incorrectos.
5. Optimizar la recursión para tableros grandes: Para evitar problemas de rendimiento en tableros extensos, considerar implementar una versión iterativa del algoritmo de expansión usando una cola en lugar de recursión pura.
6. Establecer un protocolo claro entre lógica e interfaz: Definir exactamente qué información devuelven funciones como `jugar_izq` y `jugar_der` para que la interfaz sepa cómo interpretar los resultados.

7. Realizar pruebas con semillas aleatorias fijas: Usar (random-seed valor) durante las pruebas para que los tableros sean reproducibles y los errores se puedan replicar.
8. Planificar el tiempo considerando la curva de aprendizaje: Asignar más tiempo a las partes nuevas o complejas, como la integración entre programación funcional e interfaz gráfica.

Bibliografía

- [1] J. E. Guzmán, *Introducción a la programación con Scheme*. Cartago, Costa Rica: Editorial Tecnológica de Costa Rica, 2006.
- [2] M. Rivera Meneses, *Presentación: Paradigma de Programación Funcional*, notas de clase, Tecnológico de Costa Rica, 2025.
- [3] OpenAI, *ChatGPT* [Online]. Available: <https://chat.openai.com/>.

Bitácora

Fecha: 06/09/2025

Participantes: Todos

Objetivo: Análisis inicial y diseño de la arquitectura del sistema.

Actividades realizadas:

Se realizó una reunión inicial donde se acordó utilizar una representación del tablero mediante listas de listas donde cada celda contendría un valor numérico (0-8 para minas adyacentes, 9 para mina). Se vio la necesidad de implementar matrices booleanas paralelas para manejar el estado de revelado y banderas. Cada celda contendría un par de valores: el primero indicando si hay mina y el segundo la cantidad de minas adyacentes. Se procedió a implementar la creación de una matriz de ceros como base, probando con tamaños pequeños para verificar su correcto funcionamiento. Se definió que la lógica de minas y adyacencia se desarrollaría en módulos separados.

Fecha: 08/09/2025

Participantes: Saddy

Objetivo: Implementar las funciones básicas de creación y visualización del tablero.

Actividades realizadas:

Se desarrollaron las funciones `crear_fila` y `crear_tablero_vacio` que construyen recursivamente una matriz de ceros con las dimensiones especificadas. Para facilitar la depuración, se implementó `imprimir_tablero` que muestra el estado del tablero en consola. Se probó con tableros pequeños (3x3, 4x4) verificando que la estructura se generaba correctamente.

Fecha: 09/09/2025

Participantes: Luis

Objetivo: Implementar la lógica de generación y colocación de minas.

Actividades realizadas:

Se creó la función `calcular_bombas` que determina el número de minas según el nivel de dificultad (fácil 10%, medio 15%, difícil 20%) usando división entera con redondeo hacia arriba. Se implementó `generar_posiciones_unicas` que garantiza posiciones aleatorias sin repetición mediante recursión y verificación de membresía. La función `colocar_bombas` recibe una lista de posiciones y modifica el tablero base.

Fecha: 11/09/2025

Participantes: Gabriel

Objetivo: Calcular el número de minas adyacentes para cada celda segura.

Actividades realizadas:

Se implementó el sistema de vecindad mediante `vecinos_de` que genera las 8 posiciones circundantes válidas. La función `concat_vecinos_de_bombas` obtiene todas las celdas adyacentes a minas, y `contar_frecuencias` cuenta cuántas minas rodean cada celda. Finalmente, `crear_tablero_completo_desde_bombas` integra todo colocando minas y actualizando los números adyacentes.

Fecha: 13/09/2025

Participantes: Luis, Saddy

Objetivo: Crear el sistema de matrices booleanas para estado del juego.

Actividades realizadas:

Se desarrollaron funciones genéricas para matrices booleanas (`crear_matriz_bool`, `obtener_en_matriz_bool`, `reemplazar_en_matriz_bool`) que servirían tanto para el estado de revelado como para las banderas. Se implementaron versiones específicas para el revelado y se probó la actualización de celdas individuales.

Fecha: 13/09/2025

Participantes: Gabriel

Objetivo: Implementar el algoritmo de expansión recursiva (flood fill).

Actividades realizadas:

Se creó la función `expandir_desde` que, al revelar una celda con valor 0, expande automáticamente a todas las celdas vecinas vacías mediante `expandir_vecinos`. El algoritmo usa una aproximación recursiva pura con verificación de límites y evita ciclos infinitos marcando celdas ya visitadas. Se probó con tableros que tenían grandes áreas vacías.

Fecha: 15/09/2025

Participantes: Saddy

Objetivo: Diseñar e implementar la interfaz gráfica básica con Racket GUI.

Actividades realizadas:

Se creó la ventana principal (frame) con paneles verticales y horizontales para los controles. Se añadieron campos de texto para filas, columnas y nivel, junto con el botón "Generar". Se implementó la función `resize_canvas_a_tablero` que ajusta dinámicamente el tamaño del canvas según las dimensiones del tablero.

Fecha: 15/09/2025

Participantes: Saddy

Objetivo: Implementar el dibujo del tablero en el canvas.

Actividades realizadas:

Se desarrolló draw_tablero que recorre recursivamente las matrices del tablero, estado de revelado y banderas. Las celdas se dibujan como rectángulos grises si están ocultas, mostrando el número o bomba según corresponda cuando están reveladas. Las banderas se representan con triángulos naranjas. Se manejaron cuidadosamente las coordenadas y el espaciado entre celdas.

Fecha: 18/09/2025

Participantes: Luis

Objetivo: Conectar los eventos de clic con la lógica del juego.

Actividades realizadas:

Se implementó la clase tablero-canvas% que hereda de canvas% y maneja eventos de ratón. La función coord->idx convierte coordenadas de píxeles a índices de matriz. Los métodos revelar_desde y toggle_bandera se conectaron con los clics izquierdo y derecho respectivamente, actualizando el estado y refrescando la visualización.

Fecha: 19/09/2025

Participantes: Gabriel

Objetivo: Refinar la generación de minas para mejor jugabilidad.

Actividades realizadas:

Se ajustó el algoritmo de generación de posiciones para evitar que la primera celda clickeada sea una mina. Esto requirió modificar generar_posiciones_unicas_aux2 para reintentar cuando la posición candidata coincidía con la celda inicial del jugador. Se mejoró la aleatoriedad usando random-seed para pruebas consistentes.

Fecha: 21/09/2025

Participantes: Todos

Objetivo: Implementar la detección de fin del juego y mecanismos de banderas.

Actividades realizadas:

Se crearon las funciones jugar_izq y jugar_der que encapsulan toda la lógica de juego y retornan un veredicto (sigue, gana, perdió). Se implementó toggle_en_matriz_bool para el marcado de banderas. La función revelar_bombas muestra todas las minas al perder. Los callbacks on_lose y on_win muestran mensajes apropiados.

Fecha: 21/09/2025

Participantes: Gabriel

Objetivo: Implementar y verificar la condición de victoria.

Actividades realizadas:

Se desarrolló es_victoria que compara recursivamente el tablero lógico con la matriz de revelado. La victoria ocurre cuando todas las celdas sin minas están reveladas. Se probó con tableros pequeños donde se revelaban manualmente todas las celdas seguras, verificando que el estado cambiaba correctamente a gano.

Fecha: 23-25/09/2025

Participantes: Todos

Objetivo: Depuración integral y optimización.

Actividades realizadas:

Se realizaron pruebas exhaustivas con diferentes configuraciones (8x10 nivel 2, 12x15 nivel 3, etc.). Se corrigió un error donde la expansión recursiva podría causar stack overflow en tableros muy grandes. Se optimizó el redibujado del canvas para mejorar el rendimiento. Se ajustaron colores y tamaños para mejor legibilidad.

Fecha: 24-27/09/2025

Participantes: Todos

Objetivo: Finalización, documentación y preparación para entrega.

Actividades realizadas:

Se dividió la documentación técnica describiendo cada función y algoritmo. Se creó el manual de usuario con instrucciones claras. Se preparó una demostración funcional para la defensa, mostrando todas las características: generación de tableros, revelado recursivo, banderas y detección de victoria/derrota.